# Testing Report

**Cohort 2 Team 9**
Charlie Armstrong
Adam Carey
Matthew Havers
Matthew Kelsey
Daniel Packer
Niall Potts
Sam Savidge
Ke Wu

Having completed the implementation section, we needed to test the program to ensure that the new functionality that had been added worked as intended, and that the base functionality had been fully integrated. To test the program, we decided to use a mixture of automated and manual testing methods to ensure full coverage of the program.

We decided to use automated testing methods to test the key aspects of our project, such as the events and characters, to allow for versatility and ease of testing. To achieve this, we used the headless testing system implemented by LibGDX, as it allows the tests to be directly integrated into the project, which means that the components can be tested directly, without the need for a graphical user interface. This ensures that all of the backend runs effectively, and independently of the interface, resulting in more modular code.

The aspects that are to be tested by the automated tests are the events, the characters, the timer, the score, the movement, and some achievements. These are suitable to be automated as they have little dependency on the graphical user interface, and require complex code to be able to run, which is vital to the running of the program as a whole.

However, other aspects of the program, such as the leaderboard, music, and screens, must be tested manually, due to their graphical nature. To test these, we decided on manual run through tests, where the program was run and each of these features were tested to ensure that they functioned correctly. These tests required a much more qualitative approach, to ensure that they were still functioning as intended.

To track the tests that have been completed, we created a traceability matrix, allowing us to mark the tests that have been completed, as well as any tests that could not be completed. The traceability matrix can be found here: Tracibility Matrix.png

1. DeanTest: These tests tested the functionality of the Dean in the program, which forms the main negative effect of the game. 5 tests were run, covering: The start position of the Dean, ensuring that the Dean was always generated in the correct position; the movement of the Dean, the Dean always moves towards the player; the Dean's position reset if the player was caught; and ensuring that the Avoid Dean achievement was processed correctly. The Dean tests had a 100% pass rate, and covered the full functionality of the Dean requirements, so no manual tests were needed.

2. GameTimerTest: These tests tested the functionality of the GameTimer in the program, which controls the length of the game. 6 tests were run, covering: Timer Initialisation, ensuring that the timer is generated correctly; Timer Increment, ensuring that the timer increases as intended, both by normal gameplay and from the events; Timer Decrement, ensuring that the Timer decreases as intended; The upper bound of the Timer, ensuring that the Timer did not exceed the time limit for the game; The lower bound; ensuring that events did not cause the Timer to go below 0; and the format for the Timer; ensuring that the time left is presented in a readable form. The GameTimer tests had a 100% pass rate, and covered the full functionality of the Timer requirements, so no manual tests were needed.

3. WinScreenTest: These tests tested the functionality of the Win Screen in the program, which is generated if the player successfully completes the game. 3 tests were run, covering: ensuring that the final score is correctly calculated; ensuring that the Achievements are successfully accounted for; and correctly formatting the screen if no Achievements have been gained. The WinScreen tests had a 100% pass rate, but did not test the graphical aspect of the Win Screen, which will need to be tested manually. Once these manual tests have been completed, the entire functionality of the Win Screen will have been tested.

4. PlayerTest: These tests tested the functionality of the Player in the program, which is how the User interacts with the game. 6 tests were run, covering: the player is generated in the correct location; the player Username is stored for the leaderboard; and that the player movement (up, down, left and right) functions as intended. The Player tests had a 100% pass rate, but did not test the graphical aspects of the player, so these must be tested manually. Once the manual tests have been completed, the entire functionality of the player, in line with the requirements, will have been tested.

5. PosEventTest: These tests tested the functionality of the Positive Events, which aid the player in completing the game. 3 tests were run, covering: collecting the key to unlock the barrier; collecting the clock to add more time; and interacting with the friend NPC. The PosEvent tests had a 100% pass rate, but did not test the graphical aspects of the positive events, which will need to be tested manually. Once the manual tests have been completed, the entire functionality of the positive events, in line with the requirements, will have been tested.

6. NegEventTest: These tests covered the negative events, which prevent the player from completing the game. 3 tests were run, covering: collecting the sleep clock which stops the player; the survey which takes time to complete; and the barrier, which prevents the player from passing if they do not have a key. The NegEvent tests had a 100% pass rate, but did not test the graphical aspects of the negative events, which will need to be tested manually.

Once the manual tests have been completed, the entire functionality of the negative events, in line with the requirements, will have been tested.

7. HidEventTest: These tests covered the hidden events. 1 test was run, covering if the bus ticket was collected to be able to complete the game. This test passed, but the other aspects of the hidden events could not be tested using automated testing, so had to be tested manually. Between this test and the manual tests, the entire functionality of the hidden events, in line with the requirements, will have been tested.

Following the automated testing, it was clear that manual tests were needed, to cover the graphical aspects of the program. To test this, we devised the following manual tests to ensure that all of the requirements were tested fully. The manual tests and their results are shown in the below table:

| Req Tested | Feature tested | Explanation of the test carried out | Expected Results | Result |
|---|---|---|---|---|
| 2 | Menu screen | Run the game | It should load up onto the menu/start screen | Pass |
| 6 | Timer display | Once the game has started, watch the timer throughout the gameplay | Timer starts at 05:00 and count down as intended | Pass |
| 8 | Final time display | Complete the game and go to the win screen | Final time is displayed correctly on the win screen | Pass |
| 16 | Exit game | Press the esc key as shown in the tutorial to exit the game | Game will exit and user will return to main menu | Pass |
| 8 | End score | Both win and lose in separate games and reach both respective screens | Whether you win or lose you will receive a score. | Fail |
| **Failed test:** this test is failed since the game was designed to only calculate a score if you win - which it does do. That being said since this doesn't hinder the requirements it doesn't need to be fixed. | | | | |
| 8 | Score breakdown | Win the game and reach the win screen | You will be able to see where elements of your score and penalties came from e.g time remaining, times caught by dean, etc. | Pass |
| 10 | Default character | Start the game | The character should load in at the correct spawn point with the correct sprite | Pass |
| 11 | Character movement | Use keyboard inputs to move player | Player can move in all directions corresponding to the inputs pressed | Pass |
| 9 | Accessibility | Play the game | The game should accommodate a diverse set of users, have subtitles, no dependencies on colours etc. | Pass |
| 1 | Tutorial pausing | Press pause on the tutorial screen | Player is able to pause in the tutorial | Fail |
| **Failed test:** the tutorial is implemented as a static image meaning pausing functionality is unnecessary. As this isn't a feature specified in the brief, we will not alter this to implement this | | | | |
| 3 | Music and sound effects | Play the game with volume up | Background music will play | Fail |
| **Failed test:** there are no sound effects or background music, as this is just an extra to the game and not specified in the brief we decided not to implement this | | | | |
| 3 | Family friendly design | Look around at all the assets and visual of the game | All visuals are family friendly | Pass |
| 3 | Reasonable difficulty | Play the game to completion | Simple to understand and enough time for | Pass |

| | | | | |
|---|---|---|---|---|
| | | | trial and error while still escaping | |
| 5 | Offline play | Turn off my internet connection and run the game | Game will still run as it would with internet | Pass |
| 4 | Settings menu | Open setting menu | Settings menu opens with options to alter sound | Fail |
| **Failed test:** as stated previously the settings screen was not implemented and as it is not stated in the brief we won't be adding this | | | | |
| 12 | Positive event | Walk around and trigger each of the positive events | All events should correctly trigger and have the intended effects. | Pass |
| 12 | Negative events | Walk around and trigger each of the negative events | All events should correctly trigger and have the intended effects | Pass |
| 12 | Hidden events | Walk around and trigger each of the hidden events | All events should correctly trigger and have the intended effects | Pass |
| 12,15 | Dean behaviour | Get close enough to dean and observe how it works | Dean should follow the player and teleport them back to spawn if he catches you | Pass |
| 13 | Leaderboard behavior | Finish 5 games to fill the leaderboard, then finish 2 more, one with a score that will replace a leaderboard score and one that won't | The first 5 scores will be displayed, then the next winning score will replace the correct score out of the 5 and the leaderboard will change accordingly, then the 7th score won't be added | Pass |
| 14 | Achievements notification | Get an achievement | Some sort of notification should tell me that I have the achievement | Fail |
| **Failed test:** when you get an achievement there is no such notification, however instead of that there is a separate screen that can be accessed from the main menu that shows all the achievements that you have. We decided that this is enough and is still in line with this requirement and the brief | | | | |
| 14 | Achievement score impact | Get some achievements and complete the game | Should see some score benefits or acknowledgments of the achievement | Pass |

Between the automated and manual tests, every User requirement has been met and tested. The functionality that produced failed tests was additional to the scope of key requirements, so does not impact the core functionality of the program, as outlined in the design brief, and the game is functional and can be run to completion without any errors occurring.